ENUNCIADO DEL PROYECTO FINAL: PIPELINE DEVOPS DE EXTREMO A EXTREMO PARA CIRCLEGUARD

Ingeniería de Software V

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
GRUPOS:	Grupo 003 Grupo 005	**PROFESORES:**	Grupo 003: Alejandro Muñoz Bravo Grupo 005: Juan José Valencia Jaramillo
PONDERACIÓN:	**Parte 1: 15.0 %** (reemplaza al Taller 3) **Parte 2: 25.0 %** (bloque Proyecto Final)	**MODALIDAD:**	Equipos de 3 estudiantes (ver Sección 3)
POLÍTICA IAG:	Nivel 3: Colaboración con IAG (con registro de prompts)	**ENTREGA:**	Parte 1: Domingo 8 de noviembre de 2026, 23:59 Parte 2: Domingo 22 de noviembre de 2026, 23:59

Resultados de aprendizaje y competencias que se evalúan

RAA	Resultado de aprendizaje del curso	Competencia (SO)	Dónde se evidencia
RAA1	Explicar las fases del ciclo de vida que generan discontinuidad Dev–Ops y los principios DevOps (*shift-left*, *shift-right*).	SO2 Diseño de ingeniería; SO3 Comunicación efectiva	Documentación, patrones, presentación (P1 y P2)
RAA2	Automatizar desarrollo, pruebas, despliegue y operación mediante pipelines de integración y entrega continua (CI/CD).	SO6 Diseño de experimentos	Pipeline y pruebas (principalmente P1)
RAA3	Desarrollar pipelines con *shift-left* y *shift-right* que mejoren el diseño arquitectónico y aseguren seguridad, eficiencia, disponibilidad y escalabilidad.	SO9 Despliegue y puesta en funcionamiento	IaC, observabilidad, seguridad (principalmente P2)

1 Contexto e Introducción del Proyecto

1.1 El producto: CircleGuard

CircleGuard es un sistema universitario de rastreo de contactos y cercos sanitarios que identifica grupos de contacto (*Circles*) y aplica cercos de salud preservando el anonimato individual. Su arquitectura es de microservicios con un modelo de datos híbrido: grafo temporal en **Neo4j** (ventana de 14 días), bóveda de identidades en **PostgreSQL**, eventos asíncronos en **Apache Kafka**, caché en **Redis** y autenticación dual con **OpenLDAP**. El repositorio base entregado por el docente es un monorepo **Gradle** (Java 21, Spring Boot 3.2) con **8 microservicios** y un cliente **Expo (React Native)** con salida web en `mobile/`.

Microservicio	Puerto	Responsabilidad
`auth-service`	8180	Autenticación dual LDAP/local y RBAC dinámico.
`identity-service`	8083	Bóveda criptográfica que anonimiza identidades reales.
`promotion-service`	–	Motor de estados (Sospechoso → Probable → Confirmado) sobre Neo4j.
`notification-service`	8082	Despacho multicanal (push, correo, SMS) a partir de eventos Kafka.
`form-service`	8086	Cuestionarios de salud dinámicos; publica eventos a Kafka.
`file-service`	8085	Almacenamiento de certificados y documentos (compatible S3).
`gateway-service`	8087	Validación de ingreso al campus con QR firmados y de vida corta (Redis).
`dashboard-service`	8084	Analítica geoespacial de focos que preserva la privacidad.

1.2 El reto de implementar DevOps

Hasta el momento CircleGuard es funcional, pero no implementa **ninguna** de las prácticas DevOps del curso: no hay *Dockerfiles*, *pipelines*, *Jenkinsfile*, infraestructura declarativa, métricas expuestas (no incluye Spring Boot Actuator ni Micrometer), patrones de resiliencia ni pruebas E2E. Además, `docker-compose.dev.yml` contiene credenciales en texto plano, válido para desarrollo local pero inaceptable para un entorno compartido. Su equipo asume el rol de la unidad de plataforma de la universidad y debe llevar CircleGuard desde el *commit* de un desarrollador hasta producción de forma automatizada, segura, observable y reproducible, apoyándose en las métricas de éxito del producto.

1.3 Estructura en dos entregas

	Parte 1: *Spin-off*, pipeline CI/CD inicial	**Parte 2: incremento, producto final y sustentación**
Principio DevOps	Automatización, colaboración, integración y entrega continua.	Parte 1 + **infraestructura como código**, observabilidad y *shift-right*.
Alcance	Del desarrollo al despliegue, pasando por empaquetamiento y pruebas. Unidades 1 a 5.	Pipeline incrementado, IaC, Kubernetes, patrones de diseño, seguridad, calidad y documentación completa.
Herramientas	Bash, nube, GitHub, Jenkins, SonarQube, Playwright.	Terraform, Kubernetes, Prometheus, Grafana, Jaeger (más lo anterior).
Peso	**15 %** de la nota final	**25 %** de la nota final

2 Metodología de Trabajo

2.1 Conformación de equipos

El proyecto se desarrolla en **equipos de 3-4 estudiantes** del mismo grupo (003 o 005). Los equipos se registran antes de iniciar el Sprint 0 y no pueden cambiar de integrantes después de la Parte 1.

2.2 Marco ágil y ciclo de trabajo

Cada equipo adopta **Scrum** o **Kanban** (a elección, justificando la decisión) y gestiona su trabajo en **GitHub Projects** (o Jira/Trello, con acceso para el docente). El trabajo se organiza en tres iteraciones:

Iteración	Periodo	Objetivo y evidencia
Sprint 0	Desde la publicación del enunciado	Conformar el equipo, definir el marco ágil y la estrategia de *branching*, *backlog* inicial con historias de usuario y criterios de aceptación, aprovisionar la nube y cargar el código base.
Sprint 1	Hasta Domingo 8 de noviembre de 2026, 23:59	Pipeline CI/CD inicial funcionando de punta a punta. *Sprint review* y retrospectiva documentadas. **Cierra con la Parte 1.**
Sprint 2	Hasta Domingo 22 de noviembre de 2026, 23:59(se recomienda iniciar la IaC en paralelo durante el Sprint 1)	Incremento: Terraform, Kubernetes, observabilidad, patrones, seguridad y documentación. *Sprint review*, retrospectiva y lecciones aprendidas. **Cierra con la Parte 2.**

2.3 Estrategia de *branching* y control de cambios

Se debe documentar y aplicar una estrategia explícita (GitFlow, GitHub Flow o *trunk-based*) con: ramas protegidas (`main`), *pull requests* con al menos una revisión de otro integrante, *checks* obligatorios del pipeline antes de fusionar y mensajes de *commit* bajo **Conventional Commits**, que alimentan el versionado semántico automático y las *release notes*.

```
feat(promotion): agrega cascada de estados con TTL de 14 dias   # incrementa MINOR
fix(gateway): corrige expiracion de QR firmado                  # incrementa PATCH
feat(auth)!: reemplaza esquema de roles                         # incrementa MAJOR
```

2.4 Trazabilidad individual y uso de IAG

Las calificaciones son grupales, pero se aplica un **factor de contribución individual** (entre 0.0 y 1.0) sobre la nota de cada integrante, según la evidencia en GitHub (*commits*, *pull requests*, revisiones), las tarjetas del tablero ágil, y la sustentación. El uso de IAG es de **Nivel 3 (Colaboración con IAG)**: se permite apoyarse en ella para completar los entregables, pero cada equipo debe declarar el nivel de uso, adjuntar como anexo el **registro de prompts** (contenido propio entregado a la IAG más los prompts empleados) y estar en capacidad de modificar y explicar todo resultado generado. Una configuración que el equipo no pueda explicar en la sustentación se califica como no evidenciada.

3 Entregables

3.1 Parte 1: *Spin-off*, implementación inicial del pipeline CI/CD (15 %)

El pipeline debe cubrir el ciclo **desarrollo** → **construcción** → **pruebas** → **empaquetamiento** → **despliegue** sobre los 8 microservicios y el cliente web, aplicando automatización, colaboración, integración y entrega continua.

Cód.	Entregable	Descripción y criterios mínimos
E1.1	Gestión ágil y *branching*	Tablero con *backlog*, historias de usuario con criterios de aceptación, Sprint 0 y Sprint 1 documentados (planeación, *review*, retrospectiva) y estrategia de *branching* con ramas protegidas.
E1.2	Infraestructura en la nube con Bash	Servidores en la nube (p. ej. AWS EC2) para Jenkins, SonarQube y el registro de artefactos (Nexus o equivalente). Entornos **dev, stage y prod** aislados. Todo el aprovisionamiento mediante *scripts* Bash idempotentes (`infra/bootstrap-*.sh`) y *Docker Compose*; no se acepta configuración manual no documentada.
E1.3	Empaquetamiento inmutable	*Dockerfile multi-stage* por microservicio y para el cliente web (`expo export`), usuario no root, imágenes etiquetadas con versión semántica (nunca `latest`) y publicadas en el registro. Un `docker-compose` por ambiente con *healthchecks*.
E1.4	Pipeline Jenkins (CI)	`Jenkinsfile` *multibranch* activado por *webhook* de GitHub, con *Shared Library* para etapas reutilizables y ejecución por servicio (modificado o todos). Etapas: *checkout*, compilación Gradle, pruebas unitarias con cobertura (JaCoCo), análisis **SonarQube** con *quality gate* bloqueante, pruebas de integración, empaquetamiento y publicación.
E1.5	Entrega continua y promoción	Despliegue automático a **dev**, promoción a **stage** tras pasar las pruebas E2E y **aprobación manual** (`input`) para **prod**. Versionado semántico automático con *tag* en Git, notificaciones automáticas de fallo (correo, Slack o Discord) y *rollback* básico redesplegando una versión anterior.
E1.6	Pruebas automatizadas	Pruebas **unitarias** en cada servicio (meta: $\geq$ 60 % de cobertura de líneas en `auth`, `identity`, `promotion`, `form` y `gateway`), al menos **3 pruebas de integración** entre servicios relacionados (p. ej. `form` → Kafka → `promotion`, con Testcontainers) y **pruebas E2E con Playwright** sobre el cliente web (mínimo 3 flujos: inicio de sesión, cuestionario de salud y validación de acceso). Informes publicados por el pipeline.
E1.7	*Change management* y *release notes*	Plantilla de solicitud de cambio, política de aprobación por tipo de cambio, *release notes* generadas automáticamente desde *commits* y *pull requests*, y esquema de etiquetado de *releases*.
E1.8	Documentación	`README` operativo, diagrama de arquitectura y del *pipeline*, bitácora de IAG (anexo).

Alcance de despliegue en la Parte 1. Para acotar la carga de las máquinas, los servicios **núcleo** (`auth`, `identity`, `promotion`, `form`, `gateway`) con su *middleware* (PostgreSQL, Neo4j, Kafka, Redis, OpenLDAP) y el cliente web deben estar **desplegados** en los tres ambientes. Los demás (`notification`, `file`, `dashboard`) deben ser **compilados, probados y empaquetados** por el pipeline y se despliegan en la Parte 2.

3.2 Parte 2: incremento del proyecto, producto final y presentación (25 %)

Se parte del pipeline de la Parte 1 y se incorpora **infraestructura como código** y las prácticas de *shift-right*. Los 8 microservicios deben quedar desplegados en Kubernetes.

Cód.	Entregable	Descripción y criterios mínimos
E2.1	IaC con Terraform	Infraestructura completa declarada con Terraform en **módulos** reutilizables (red, cómputo/clúster, registro, almacenamiento, observabilidad), **tres ambientes** (dev, stage, prod) parametrizados, *backend* remoto del estado con bloqueo (p. ej. S3 + DynamoDB) y diagramas de la infraestructura.
E2.2	Orquestación en Kubernetes	Despliegue de los 8 servicios y el *middleware* con manifiestos, Helm o Kustomize: *namespaces* por ambiente, *ConfigMaps* y *Secrets*, *Ingress*, *HPA* y sondas *liveness*/*readiness* alimentadas por Actuator. Se admite EKS o un clúster ligero (k3s) sobre la nube.
E2.3	Pipeline incrementado	Jenkins ejecuta `terraform plan/apply` (con aprobación en prod), construye, escanea con **Trivy** (imágenes e IaC) con umbral bloqueante, despliega a Kubernetes, promueve entre ambientes con aprobación a producción y notifica fallos.
E2.4	Patrones de diseño	Identificar y documentar los patrones presentes en la arquitectura (p. ej. *database-per-service*, arquitectura dirigida por eventos, *API Gateway*, *cache-aside*, máquina de estados; verificar en el código). Implementar o mejorar **al menos tres**: uno de **resiliencia** (*Circuit Breaker*, *Retry*, *Bulkhead* con Resilience4j), uno de **configuración** (*External Configuration*, *Feature Toggle*) y uno adicional a elección. Cada uno con propósito, *trade-offs*, evidencia y prueba.
E2.5	Observabilidad	Métricas con Micrometer y **Prometheus**; **Grafana** con *dashboards* por servicio y de negocio (p. ej. tiempo de contención, promociones por minuto, *check-ins*); **al menos 3 alertas críticas** (Alertmanager); **trazado distribuido con Jaeger** (OpenTelemetry) a través de `form` → Kafka → `promotion` → `notification`; y gestión centralizada de logs (ELK o alternativa como Loki).
E2.6	Seguridad	Escaneo continuo de vulnerabilidades (Trivy, dependencias), gestión segura de secretos (sin credenciales en el repositorio), RBAC en Kubernetes y en Jenkins, y **TLS** en los servicios expuestos.
E2.7	Pruebas avanzadas	Pruebas de rendimiento y estrés con **Locust** (escenarios sobre validación de acceso y envío de cuestionarios, con *p95*, *throughput* y tasa de error frente a un objetivo declarado), **OWASP ZAP** (*baseline scan*) en el pipeline e informe consolidado de cobertura y calidad.
E2.8	*Change management* avanzado	*Release notes* de cada versión publicada, plan de *rollback* **ejecutado y evidenciado** en Kubernetes y registro de cambios por ambiente.
E2.9	Documentación completa	Arquitectura con diagramas, metodología ágil aplicada (*burndown* o flujo acumulado, retrospectivas), patrones, infraestructura como código, análisis de resultados de pruebas, **costos** de infraestructura (estimación y real), **manual de operaciones** y *resumen ejecutivo en inglés* (1 página).
E2.10	Presentación	Sustentación (arquitectura e infraestructura, demostración de CI/CD, aplicación funcionando, *dashboards*, resultados de rendimiento, lecciones aprendidas).

Bonificaciones (opcionales, solo Parte 2). Multi-cloud, *service mesh* (Istio/Linkerd con mTLS y *canary*), *chaos engineering* (Chaos Mesh/Litmus) y FinOps (monitoreo de costos, *spot*, *scale-to-zero*). Cada una aporta hasta **+0.10** sobre la nota de la Parte 2 (escala 0.0 a 5.0), con un tope acumulado de **+0.25**; la nota máxima sigue siendo 5.0. Se evalúan solo si los entregables obligatorios están completos.

3.3 Estructura sugerida del repositorio

```
circleguard/
|-- services/ ... mobile/                # codigo base (no se reescribe)
|-- Jenkinsfile  jenkins/vars/           # pipeline y Shared Library
|-- infra/bootstrap/                     # P1: scripts Bash (Jenkins, SonarQube, Nexus)
|-- infra/terraform/{modules,envs/{dev,stage,prod}}   # P2: IaC
|-- deploy/{compose,k8s}/                # P1: compose por ambiente / P2: Kubernetes
|-- tests/{e2e-playwright,performance-locust,security-zap}/
|-- observability/{prometheus,grafana,alerts}/
|-- docs/{arquitectura,agil,patrones,operaciones,costos,release-notes}/
|-- CHANGELOG.md  README.md  IAG_REGISTRO.md
```

4 Fechas de Entrega

Entrega	Fecha	Condiciones
Parte 1 Semana 14	Domingo 8 de noviembre de 2026, 23:59	Repositorio, tablero ágil, *pipeline* en funcionamiento (accesible al docente), informe PDF con enlaces y evidencias, anexo de IAG, cargados en la plataforma.
Parte 2 Semana 16	Domingo 22 de noviembre de 2026, 23:59	Se entrega documentación completa, y presentación; luego se realiza la sustentación más preguntas.

Los entregables sin evidencia verificable (capturas, ejecuciones en el *pipeline*, repositorio) no se califican en el criterio correspondiente.

5 Evaluación

5.1 Distribución del peso entre las dos partes

El borrador original asigna 100 puntos a nueve componentes. Esos puntos se escalan al peso real del proyecto (15 % + 25 % = 40 % de la nota final del curso) y se reparten según las herramientas y unidades que corresponden a cada entrega.

Componente	Equivalente en la nota final	Parte 1	Parte 2
Metodología ágil y *branching*	4.0 %	2.0 %	2.0 %
Infraestructura (Bash/nube en P1; Terraform y Kubernetes en P2)	8.0 %	2.0 %	6.0 %
Patrones de diseño	4.0 %	–	4.0 %
CI/CD avanzado	6.0 %	5.0 %	1.0 %
Pruebas completas	6.0 %	3.5 %	2.5 %
Change management y *release notes*	2.0 %	1.5 %	0.5 %
Observabilidad y monitoreo	4.0 %	–	4.0 %
Seguridad	2.0 %	–	2.0 %
Documentación y presentación	4.0 %	1.0 %	3.0 %
TOTAL	**40.0 %**	**15.0 %**	**25.0 %**

Cálculo. Cada criterio se califica de 0.0 a 5.0 y se pondera con su peso. La nota de cada parte (0.0 a 5.0) es la suma de las notas ponderadas dividida entre el peso total de esa parte (15 o 25), y luego se multiplica por el factor de contribución individual. Ejemplo: una nota de 4.0 en la Parte 1 aporta $4{,}0 \times 0{,}15 = 0{,}60$ puntos a la nota final del curso.

5.2 Rúbrica analítica de la Parte 1 (15 % de la nota final)

Criterio (entregable)	Peso	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
P1.1 Gestión ágil y *branching* (E1.1)	**2.0 %**	Tablero vivo con historias y criterios de aceptación medibles; Sprint 0 y 1 con *review* y retrospectiva; ramas protegidas y PR revisados por pares.	Tablero y estrategia definidos, pero con historias vagas, retrospectiva superficial o PR sin revisión consistente.	Sin tablero verificable, sin estrategia de *branching* o trabajo directo sobre `main`.
P1.2 Nube y automatización con Bash (E1.2)	**2.0 %**	Tres ambientes aislados; *scripts* idempotentes que reconstruyen Jenkins, SonarQube y registro desde cero con un comando.	Ambientes operativos, pero con pasos manuales documentados o *scripts* no idempotentes.	Infraestructura configurada a mano sin *scripts*, o ambientes no accesibles.
P1.3 Empaquetamiento inmutable (E1.3)	**2.5 %**	Imágenes *multi-stage*, no root, versionadas y publicadas en el registro para los 8 servicios y el cliente; *compose* con *healthchecks*.	Imágenes funcionales pero pesadas, con *tags* ambiguos o para solo una parte de los servicios.	Sin imágenes reproducibles, uso de `latest` o artefactos no publicados.
P1.4 Pipeline CI en Jenkins (E1.4)	**2.5 %**	*Multibranch* con *webhook*, *Shared Library*, JaCoCo y *quality gate* de SonarQube que bloquea; ejecución selectiva por servicio.	Pipeline funcional con *Jenkinsfile* monolítico, *quality gate* informativo o sin *webhook*.	El pipeline falla, solo compila o no integra análisis estático.
P1.5 Entrega continua y promoción (E1.5)	**2.5 %**	Despliegue automático a dev, promoción a stage tras E2E, aprobación manual a prod, semver automático, notificaciones de fallo y *rollback* demostrado.	Despliega en los ambientes, pero con versionado manual, sin aprobación formal o sin notificaciones.	Despliegue manual o ambientes no promovidos por el pipeline.
P1.6 Pruebas automatizadas (E1.6)	**3.5 %**	Cobertura $\geq$ 60 % en servicios núcleo, $\geq$ 3 integraciones relevantes y 3 flujos E2E con Playwright, todo ejecutado y reportado por el pipeline.	Pruebas presentes pero con cobertura baja, integraciones triviales o E2E ejecutadas fuera del pipeline.	Pruebas ausentes, no ejecutables o que no se integran al pipeline.
P1.7 *Change management* y *release notes* (E1.7)	**1.5 %**	Proceso formal con plantilla y política de aprobación; *release notes* automáticas y etiquetado coherente con semver.	Proceso descrito pero *release notes* manuales o etiquetas inconsistentes.	Sin proceso de cambios ni *release notes*.
P1.8 Documentación, video e IAG (E1.8)	**1.0 %**	`README` que permite reproducir, diagramas claros, y bitácora de IAG completa con reflexión crítica.	Documentación parcial, bitácora de IAG incompleta.	No se puede reproducir, sin registro de IAG.
TOTAL PARTE 1	**15.0 %**	**Nota para aprobar $\geq$ 3.0**		

5.3 Rúbrica analítica de la Parte 2 (25 % de la nota final)

Criterio (entregable)	Peso	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
P2.1 Metodología ágil, cierre (E2.9)	**2.0 %**	Sprint 2 con *review*, retrospectiva y métricas (*burndown*/flujo) que muestran mejoras entre iteraciones.	Iteraciones documentadas sin análisis de mejora.	Sin evidencia de la segunda iteración.
P2.2 Terraform (E2.1)	**4.0 %**	Módulos reutilizables, 3 ambientes por variables, *backend* remoto con *lock*, `plan` limpio y diagramas coherentes con lo desplegado.	Terraform funcional pero monolítico, con ambientes duplicados o sin *lock* del estado.	Infraestructura no declarada, estado local o `apply` fallido.
P2.3 Kubernetes (E2.2)	**2.0 %**	Los 8 servicios y el *middleware* en K8s por ambiente, con *Secrets*, Ingress, HPA y sondas útiles.	Despliegue parcial o sin sondas, HPA o separación de ambientes.	No hay servicios corriendo en el clúster.
P2.4 Patrones de diseño (E2.4)	**4.0 %**	Patrones existentes identificados con evidencia en código; $\geq$ 3 nuevos (resiliencia, configuración y otro) implementados, probados (p. ej. falla inyectada) y con *trade-offs* analizados.	Patrones implementados pero sin pruebas de su efecto, o análisis superficial.	Menos de 3 patrones, sin implementación real o solo descriptivos.
P2.5 CI/CD avanzado (E2.3)	**1.0 %**	Terraform y K8s dentro del pipeline, Trivy bloqueante, aprobación a prod y notificaciones.	Pipeline despliega a K8s pero sin Trivy bloqueante o con pasos manuales.	El pipeline no gobierna el despliegue en K8s.
P2.6 Pruebas avanzadas (E2.7)	**2.5 %**	Locust con escenarios realistas y objetivos declarados, ZAP en el pipeline, informe que interpreta resultados y propone mejoras.	Pruebas ejecutadas sin objetivo claro o sin análisis.	Sin pruebas de rendimiento o seguridad.
P2.7 *Change management* y *rollback* (E2.8)	**0.5 %**	*Release notes* por versión y *rollback* ejecutado y evidenciado en K8s.	Plan escrito sin ejecución comprobada.	Sin plan de *rollback*.
P2.8 Observabilidad (E2.5)	**4.0 %**	Prometheus, Grafana (técnicos y de negocio), $\geq$ 3 alertas críticas probadas, trazas Jaeger de extremo a extremo y logs centralizados.	Métricas y *dashboards* básicos; sin trazas completas o alertas sin probar.	Sin métricas expuestas o *dashboards* vacíos.
P2.9 Seguridad (E2.6)	**2.0 %**	Escaneo continuo, secretos fuera del repositorio, RBAC aplicado y TLS verificable.	Algunas prácticas aplicadas; credenciales aún en archivos o TLS autofirmado sin justificar.	Credenciales expuestas, sin RBAC ni TLS.
P2.10 Documentación completa (E2.9)	**2.0 %**	Documentación coherente con lo desplegado: arquitectura, IaC, patrones, resultados, costos, manual de operaciones y resumen en inglés.	Documentación presente pero desactualizada o con costos sin sustento.	Documentación incompleta o inconsistente con la solución.
P2.11 Presentación (E2.10)	**1.0 %**	Ver rúbrica de presentación (siguiente tabla).	Ver rúbrica de presentación.	Ver rúbrica de presentación.
TOTAL PARTE 2	**25.0 %**	**Nota para aprobar $\geq$ 3.0**		

5.4 Rúbrica de la presentación y demostración (criterio P2.11, 1.0 % de la nota final)

Dimensión	Peso	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
Arquitectura e infraestructura	**0.20 %**	Explica decisiones, *trade-offs* y diagramas con claridad.	Describe componentes sin justificar decisiones.	Descripción confusa o incorrecta.
Demostración de CI/CD en vivo	**0.25 %**	Un cambio recorre el pipeline completo hasta producción con aprobación y notificaciones, sin intervención manual oculta.	Demostración parcial o con pasos manuales.	No se evidencia el pipeline.
Aplicación y *dashboards*	**0.25 %**	Aplicación funcionando en K8s; *dashboards*, alertas y trazas interpretados con una falla provocada.	Aplicación y *dashboards* mostrados sin interpretación.	Aplicación o monitoreo no operativos.
Resultados y lecciones aprendidas	**0.15 %**	Resultados de rendimiento analizados frente a objetivos; lecciones y recomendaciones concretas.	Resultados reportados sin análisis crítico.	Sin resultados o conclusiones.
Comunicación y dominio individual	**0.15 %**	Tiempo respetado (20–30 min), introducción en inglés de 2–3 min y respuestas sólidas de todos los integrantes.	Desbalance entre integrantes o respuestas parciales.	No se respeta el tiempo o no hay dominio individual.
TOTAL PRESENTACIÓN	**1.0 %**	**La nota de este criterio (0.0 a 5.0) se promedia con los pesos anteriores**		

INSTRUCCIONES DE ENTREGA

1. **Repositorio en GitHub:** privado, en la organización indicada por el docente, con acceso al profesor de su grupo, *pipeline* en verde y *tags* de *release* (`v1.0.0`, ...).
2. **Informe en PDF** (uno por parte), con el enlace al repositorio y al tablero ágil, capturas de las ejecuciones del *pipeline*, URL de los ambientes y evidencias de cada entregable.
3. **Anexo de IAG:** nivel de uso declarado y registro de prompts (contenido propio y prompts empleados).
4. **Credenciales y costos:** no se incluyen secretos en el repositorio ni en los PDF. Los equipos deben apagar o destruir los recursos de nube una vez finalizada la evaluación (`terraform destroy` en Parte 2).